

Running your PyTorch training on Sol's Gaudi2 (HPU) nodes

A step-by-step port from the A100 container repo

Jarod DeFilippo (with Oracle)

2026-10-06

Who this is for

You are on the lab account `grp_jliang12`, you train on Sol's A100s, and your code is a fork of the course's `2026Fall-CSE507-Ark-Container` repo: an A100 image def, a wrapper around `apptainer exec, run_*/resume_*` sbatch scripts, training code in `base/`. This guide ports that fork to Gaudi2 step by step, with your repo open. It does not port your code. Each step says what to do, why, how to tell it worked and what failure looks like, so you can debug your own port.

The baseline is the lab's `Gaudi_Demo` (github.com/jlianglab/Gaudi_Demo). The files you copy come from Jarod's copy of the container repo, which ran Ark+ on 4 cards in one 7-day job. Jarod's UniMiSS+ port, an unrelated released codebase run on one card in 2D and 3D, is the second worked example. Every number was measured in those repos or in Jarod's session notes (Oct 4-6, 2026) and carries its job or node.

The map: your A100 files and their Gaudi counterparts

A100 file you have	Gaudi counterpart (Jarod's repo)	What differs	Copy or edit
In <code>containers/nvidia/</code> : <code>apptainer.def</code> , <code>requirements.txt</code> , <code>constraints.txt</code> none (<code>nvidia.sif</code> is downloaded)	In <code>containers/audi/</code> : <code>apptainer.def</code> , <code>requirements.txt</code> , <code>requirements_nodeps.txt</code> <code>build_gaudi_image.sh</code>	Habana base image, not CUDA plus PyPI torch; no venv; pip constrained to the base's own packages; no torch, torchvision or numpy listed You build the image yourself, on a gaudi node	Edit the requirements (Step 2) Edit paths and header (Step 2)
<code>nvidia.sh</code>	<code>gaudi.sh</code>	No <code>--nv</code> ; maps Slurm's card index to a Habana module ID; binds <code>/dev/shm</code> and a Habana log dir; lazy mode on	Copy, then edit its paths (Step 4)
none	<code>gaudi_hlsmi.sh</code> , <code>gaudi_check_cards.sh</code> , <code>gaudi_verify_busy.sh</code>	Card-isolation checks the A100 side never needed	Copy; the card check calls <code>./gaudi.sh</code> by name
none	<code>gaudi_smoke.sh</code>	1-card, 45-minute test of the whole path	Edit experiment, run dir, weights (Step 5)
<code>run_*.class.sh</code> , <code>resume_*.class.sh</code>	<code>run_d1_8a_gaudi.sh</code> , <code>resume_d1_8a_gaudi.sh</code>	Header, lazy mode, workers, card checks, <code>gaudi</code> argument	Edit (Steps 6-7)

A100 file you have	Gaudi counterpart (Jarod's repo)	What differs	Copy or edit
<code>common.sh</code> in <code>experiment_scripts/</code>	same file	Already has <code>gaudi</code> branches	Nothing
<code>base/*.py</code>	same files	Already route through the HPU path in <code>base/accelerator.py</code>	Nothing, unless you wrote new loops (Step 3)

The last two rows describe Jarod's copy: before the port began, `base/accelerator.py` already had an HPU path and `common.sh` already had `gaudi` branches pointing at a `gaudi.sh` that did not yet exist (Oct 4 notes). Confirm your fork matches with `grep -n hpu base/accelerator.py experiment_scripts/common.sh`.

UniMiSS+ used the same set under other names: `sol_gaudi/unimiss_gaudi.sh` is `gaudi.sh` plus a `GAUDI_PWD` option and a `/scratch` bind, and its three check scripts are Ark's, verbatim except for the wrapper's name.

Step 0: Is Gaudi right for this job?

Access. Only the lab account: `-A grp_jliang12 -p gaudi -q public`. The class account has no Gaudi QOS (`-q class_gaudi` returns "Invalid qos specification", Oct 4). QOS public caps CPUs (7,500 per user), not cards.

Hardware. Nodes `gaudi[001-010]` (004 drained on Oct 4), 8 HL-225 (Gaudi2) cards per node, about 18 cores per card (RC docs, as recorded in the notes), partition limit 7 days. Request cards with `-N 1 --gres=gpu:hl225:N`; the demo's `-N 1 -G N` is equivalent.

Queue. On Oct 4 a 1-card `sbatch --test-only` projected an immediate start, while the same account's A100 jobs projected Nov 5 on `public` and about 10.5 hours on `htc`. It is not always empty: on Oct 5 a build waited on Priority and a 1-card run waited about 34 minutes. Run `sbatch --test-only` with your real header first.

Speed. One Gaudi2, lazy mode, fp32, ViT-B/16 (DINOv3) at batch 200: 0.62 s per step (about 320 img/s) against 1.76 s on one A100-80, about 2.8x per card (smoke 64613789, gaudi003, Oct 4). The demo's ChestMNIST ResNet-50 run took 17:17:03 in lazy mode against 29:34:31 on an A100. UniMiSS+ ran 1.01 s/step in 3D (batch 8) and about 0.19 s/step in 2D (batch 32) on one card (Oct 5); no A100 timing exists for those.

Accuracy. In the demo, Ark+ in lazy mode matched the official result (VinDr-CXR student AUC 0.9412 against 0.9414); eager mode gave 0.8482 on the same run, with no error raised. Lazy mode is not optional.

Limits (demo README): no `SyncBatchNorm` (batch-norm models with small per-card batches may degrade on several cards); Gaudi's `torch.compile` lacks many ops; 3D data "seems" less memory-efficient (unconfirmed). Every measured run here is fp32; mixed precision was not tried.

For scale: the Ark+ port went from first image build to a running 4-card job in one day (Oct 4), most of it spent on card isolation (Step 4).

Step 1: Get a card and look at the node

Do. Open an interactive allocation (the TA's example, plus the account) and read the card table before writing any script:

```
salloc -A grp_jliang12 -q public -p gaudi -c 18 -G 1 --mem=64G -t 1-0
env | grep -E '^(HABANA|SLURM_JOB_GPUS|SLURM_GPUS_ON_NODE|GPU_DEVICE_ORDINAL)'
hl-smi -Q index,module_id,memory.used -f csv
```

Why. The partition refuses any job without a card (“Requested node configuration is not available”, Oct 4), and the rest of the port depends on this table.

What you should see (smoke 64599859, gaudi001, Oct 4): `SLURM_JOB_GPUS=4`, `SLURM_GPUS_ON_NODE=1`, `GPU_DEVICE_ORDINAL=0`, and no `HABANA_VISIBLE_*` at all. `hl-smi` lists all eight cards: idle ones at 768 MiB, other users’ up to 98,304 MiB at 100% compute. `SLURM_JOB_GPUS` is your card’s index on the node; `GPU_DEVICE_ORDINAL` is not (0 for index 4). `index` and `module_id` differ; gaudi001’s map:

index	0	1	2	3	4	5	6	7
module_id	2	6	3	7	0	4	1	5

gaudi003 differs at indices 5 and 6 (5 to 1, 6 to 4), so the map must be read on each node, every job.

Not verified: whether a `salloc` shell exports `SLURM_JOB_GPUS` as a batch job does; every measurement was a batch job. `gaudi.sh` maps cards only when that variable is set and runs unrestricted otherwise. Use the shell to look; train through `sbatch`.

Step 2: Build your image

The mental model. Habana ships PyTorch as its own build inside a Docker base image (`pytorch-installer-2.10.0` for Ubuntu 22.04 from `vault.habana.ai/gaudi-docker/1.24.0/`, the `From:` line of every `def` here), which Apptainer turns into a `.sif`. Your A100 image (a CUDA 13 runtime with PyPI torch 2.11.0 in a `venv`) cannot drive a Gaudi card. The base ships (build 64599857, Oct 4):

Component	Version in the Habana base
torch	2.10.0a0+gitc1e5ed4 (Habana’s build, not a PyPI wheel)
torchvision	0.25.0+cpu
numpy	2.2.6
habana packages	1.24.0.1007
Python	3.10

The one rule: pip must never replace that torch (“NVIDIA packages will corrupt the Gaudi-specific torch packages”, demo README). Two ways to enforce it:

- *Demo:* a `uv venv --system-site-packages`, so the `venv` sees the base’s torch; packages whose dependencies would pull torch or NVIDIA libraries (found with `pip install --dry-run`) go in `requirements_nodeps.txt`, installed `--no-deps`. Fine for a short, dry-run list.
- *Ark and UniMiSS:* no `venv`; freeze what the base ships and pass it to pip as a constraint, so a requirement wanting another torch or numpy fails the build instead of installing it. Better for longer lists; the cost is that a package needing, say, numpy below 2 fails until you pin a compatible version.

```
# containers/gaudi/apptainer.def, %post (excerpt)
python3 -m pip list --format=freeze > /opt/base-constraints.txt
python3 -m pip install --no-cache-dir -c /opt/base-constraints.txt \
    -r /opt/requirements.txt
python3 -m pip install --no-cache-dir --no-deps -c /opt/base-constraints.txt \
    -r /opt/requirements_nodeps.txt
python3 -c "import torch, torchvision, timm, cv2, albumentations, pydicom; \
    print(torch.__version__)"
python3 -m pip freeze > /opt/pip-freeze.txt
```

Edit your requirements, going from the A100 `requirements.txt` in `containers/nvidia/` to the Gaudi one in `containers/gaudi/`:

- Delete torch, torchvision and numpy: the base provides them, and the constraints would fight any pin.
- Move `timm==0.5.4` to `requirements_nodeps.txt`: the demo and Ark both install it `--no-deps`.
- Pin data-path libraries to the A100 versions. Unpinned, pip pulled `opencv-python-headless 5.0.0.93` and `Pillow 12.3.0` (build 64599857) against the A100 image's 4.12.0.88 and 11.3.0, so a Gaudi-versus-A100 comparison would change two things at once. `opencv 4.12.0.88` accepts `numpy 2.2.6`.
- Let `scipy float`, because the A100's `scipy 1.16.1` needs Python 3.11 (the build got 1.15.3); `tqdm` too, since the base ships it.
- Starting from the demo's `def`, delete its `export HABANA_VISIBLE_DEVICES=all` (line 11 of the `%environment` block). `gaudi_check_cards.sh` refuses a host-side `HABANA_VISIBLE_DEVICES=all`, but a value baked into the image is not visible to that check, and `gaudi.sh` sets `HABANA_VISIBLE_MODULES` without touching `HABANA_VISIBLE_DEVICES`. Which of the two the Habana runtime obeys when both are set was not tested, so removal is the only safe fix. Verify inside a job with `./gaudi.sh env | grep HABANA`: expect your `HABANA_VISIBLE_MODULES` and no `HABANA_VISIBLE_DEVICES=all`.

Build on a gaudi node with one card. Copy the header from UniMiSS's `build_image.sh`; Ark's `build_gaudi_image.sh` still says `-p htc`, and Jarod overrode it at submission (build 64599857 ran with `-p gaudi -t 02:00:00 --gres=gpu:hl225:1 -c 18`).

```
# sol_gaudi/build_image.sh (excerpt; user paths generalized)
#SBATCH -A grp_jliang12
#SBATCH -p gaudi
#SBATCH -q public
#SBATCH -N 1
#SBATCH --gres=gpu:hl225:1
#SBATCH -c 18
#SBATCH --mem=32G
#SBATCH -t 02:00:00
export APPTAINER_TMPDIR=/scratch/$USER/.apptainer/tmp
export APPTAINER_CACHEDIR=/scratch/$USER/.apptainer/cache
mkdir -p "$APPTAINER_TMPDIR" "$APPTAINER_CACHEDIR"
apptainer build --fakeroot --force \
    sol_gaudi/.unimiss_gaudi.building.sif sol_gaudi/apptainer.def
mv -f sol_gaudi/.unimiss_gaudi.building.sif sol_gaudi/unimiss_gaudi.sif
```

The card goes unused, but the partition refuses a cardless job, and on Oct 4 this route started at once where `htc` projected about 10.5 hours. `--fakeroot` prints “User not listed in `/etc/subuid`, trying root-mapped namespace” and carries on, which suffices for a pip-only `def`; whether the demo `def`'s `apt-get` works in that mode was not tested. Apptainer's `tmp` and `cache` go on `/scratch`, as in the demo's wrapper, so the build's working files stay out of your home directory (the image alone is 1.75 GB). The temp-name-then-rename keeps a failed rebuild from clobbering an image a running job uses.

Measured: Ark builds took 24:08 (64599857, `gaudi003`) and 22:12 (64607981, `gaudi007`) for a 1.75 GB image; the UniMiSS build took about 7 minutes for 1.76 GB (64699439, Oct 5), for reasons not investigated.

Check. This must show `torch==2.10.0a0+git...`, `numpy 2.2.6` and `habana 1.24.0.x`:

```
apptainer exec <image>.sif \
    grep -iE '^(torch|torchvision|numpy|habana)' /opt/pip-freeze.txt
```

Failure looks like any other torch or numpy there: stop. The pip warning “`torchvision 0.25.0+cpu` requires `torch==2.10.0`, but you have `torch 2.10.0a0+gitc1e5ed4`” comes from the base itself and is harmless.

Step 3: Make the smallest change set in your code

If your fork trains through `base/accelerator.py`, the HPU path exists: with `PROJECT_ACCELERATOR=hpu` it imports `habana_frameworks.torch.core`, uses `torch.device("hpu")`, initializes DDP with `HCCL`, exposes `accelerator.mark_step()`, and enables `pin_memory` and `cudnn.benchmark` only for `CUDA`. Keep new code

on that path. A codebase without such a layer needs the list below; UniMiSS+ is the example (commits 6898e58, 7e877db).

Change	Why	Ark (course repo)	UniMiSS+ (added)
Device <code>hpu</code> , with CUDA/CPU fallback	HPU is its own device type; the fallback keeps one codebase runnable on both images	Accelerator	hpu_device.py
<code>.cuda()</code> to <code>.to(device)</code>	a Gaudi node has no CUDA device	not needed	model, loss, inputs, labels
Import <code>htcore</code>	the module <code>habana_frameworks.torch.core</code> provides <code>mark_step</code> ; every source imports it before using the device	yes	yes
<code>mark_step()</code> after <code>backward()</code> and after <code>optimizer.step()</code>	lazy mode accumulates ops into a graph; <code>mark_step</code> cuts and runs it	yes	both, after review
<code>mark_step()</code> per eval batch, after an EMA teacher update, before a checkpoint save	the same, outside the train step	yes	eval only
DDP backend <code>hccl</code> , no <code>device_ids</code>	demo README: HPU initializes differently from CUDA	yes	n/a (1 card)
<code>torch.load</code> with <code>map_location="cpu"</code> ; <code>weights_only=False</code> only for trusted files (see below)	torch 2.6 changed the <code>weights_only</code> default to <code>True</code> ; the base ships 2.10	False already	False on 5 pretraining loads
Save CPU copies of state dicts	demo README rule (reason not stated); the file becomes device-neutral	<code>_copy_to_cpu</code>	every <code>torch.save</code>
<code>pin_memory</code> only on CUDA	<code>True</code> on HPU is unverified (the demo's BenchmarkTransformers kept it and ran)	yes	after review
<code>np.int</code> to <code>int</code>	the base ships numpy 2.2.6	not needed	2 lines

For the `torch.load` row, the rule is single and has no exception for third-party files: load anything you did not produce yourself with the restricted loader (`weights_only=True`, the default since torch 2.6), because unpickling a file runs arbitrary code as you. If the restricted loader refuses a class the checkpoint needs, allowlist that class with `torch.serialization.add_safe_globals([...])` and keep the restricted loader; the usual offenders are numpy's `scalar` and `dtype`. `weights_only=False` is for checkpoints your own code wrote, nothing else. A matching SHA-256 proves you have the file the publisher posted, not that the file is safe to unpickle, so it does not change the rule. Both worked examples predate it: Ark+ loads every checkpoint with `weights_only=False`, including the downloaded DINOv3 weights, and the UniMiSS+ port does the same on its five loads of the authors' `UniMissPlus.pth`. Neither needs to: in a check on the Mac on Oct 6 (torch 2.12), both files loaded under the restricted loader, `UniMissPlus.pth` after allowlisting numpy's `scalar` and `dtype`. Treat those `False` flags as debt to remove in your fork, not as a pattern to copy.

The `torch.load` and numpy rows are not Gaudi-specific: the A100 image also ships torch 2.11.0 and numpy 2.2.6, so they can bite older released code on either image, as does library drift (UniMiSS+ needed `from batchgenerators.transforms.abstract_transforms import Compose`). `cuda` flags are harmless: the demo sets `cuda.benchmark = True` unconditionally, UniMiSS+ kept its `cuda` lines, and both ran. Resume with `map_location=device` works on HPU (job 64620656, Oct 4: learning rate and global step continued correctly).

UniMiSSPlus/Downstream/hpu_device.py (whole file, docstring dropped, reflowed)

```

import torch

try:
    import habana_frameworks.torch.core as htcore
    device, mark_step = torch.device('hpu'), htcore.mark_step
except ImportError:
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    mark_step = lambda: None # no-op off HPU

# Gaudi_Demo/BenchmarkTransformers/trainer.py (excerpt): where mark_step goes
loss.backward()
if args.lazy_mode:
    htcore.mark_step()
optimizer.step()
if args.lazy_mode:
    htcore.mark_step()

```

Ark gives every DDP rank plain `torch.device("hpu")` and lets the runtime hand each rank one visible card; the CUDA path's `torch.cuda.set_device(local_rank)` has no counterpart in its HPU branch.

Check. Print the resolved device and torch version at startup (UniMiSS+ prints `device: hpu, torch 2.10.0a0+...`) and have the smoke test assert it. **Failure looks like** a fallback resolving to `cpu` because the Habana import failed: slow, not broken, so nothing errors unless you assert.

Step 4: Write your launch wrapper, with card isolation built in

`gaudi.sh` is `nvidia.sh` with these changes:

- No `--nv`, Apptainer's NVIDIA GPU flag; both Gaudi wrappers omit it.
- `--cleanenv` stays, so the container environment is identical every run and card selection arrives only through forwarded variables. Consequence: `SLURM_JOB_GPUS` never reaches the container, so mapping happens on the host, before `apptainer exec`. (The demo's wrapper has no `--cleanenv`.)
- It forwards `PT_HPU_LAZY_MODE` (default 1), `HABANA_VISIBLE_MODULES`, `HABANA_VISIBLE_DEVICES` and `OMP_NUM_THREADS` instead of `CUDA_VISIBLE_DEVICES`, because `--cleanenv` drops the rest. Habana's default is eager mode (demo README), so the lazy-mode setting must arrive explicitly.
- `PROJECT_ACCELERATOR=hpu`, which `base/accelerator.py` and `main_ark.py` read; no `PATH` override, since there is no `venv`.
- Binds `/dev/shm` and a Habana log directory (`.container-runtime/gaudi/habana_logs` on the host), as the demo does; no measured run went without them.

Then edit its paths for your repo and storage: the `IMAGE=` default, `--pwd /workspace/base`, the data bind (`DATA_BIND`, default `/data:/data:ro`) and `RUNTIME_ROOT` (`tmp`, `caches`, Habana logs); `unimiss_gaudi.sh` has `IMAGE=` and `RUNTIME_ROOT` too, takes the working directory from `GAUDI_PWD` (default `/workspace`) instead of a fixed `--pwd`, binds `/scratch` and `/data` with fixed lines instead of `DATA_BIND`, and adds `REPO=` (the clone root, one level above `sol_gaudi/`).

Launch with `python -m torch.distributed.run --standalone --nproc-per-node N`, because `torchrun` is not implemented on Gaudi (demo README). Take `N` from the Slurm allocation, never from `torch.hpu.device_count()`, for the reason below.

The card-isolation trap

Measured on Sol (Oct 4), these facts mean an unmodified job can train on someone else's card:

1. Slurm exports only `SLURM_JOB_GPUS`, `SLURM_GPUS_ON_NODE` and `GPU_DEVICE_ORDINAL`; no `HABANA_VISIBLE_*`.
2. Device cgroups do not hide other cards: a 1-card job's container saw 8.

3. `torch.hpu.device_count()` reports 8 even with `HABANA_VISIBLE_MODULES` set (job 64607982, gaudi003), so a count proves nothing.
4. `hl-smi`'s `index` is not the `module_id`, and the map differs per node (Step 1).
5. Habana does honor `HABANA_VISIBLE_MODULES`; only the count ignores it (proof below).

So the wrapper reads the node's map at job start, translates the allocated index to module IDs, exports `HABANA_VISIBLE_MODULES`, and refuses to run if any part fails:

```
# gaudi.sh (excerpt; the real file validates every field and refuses on any failure)
if [[ -z "${HABANA_VISIBLE_MODULES:-}" ]] && -z "${HABANA_VISIBLE_DEVICES:-}" \
    && -n "${SLURM_JOB_GPUS:-}" ]; then
    map=$(hl-smi -Q index,module_id -f csv | awk -F, '{ gsub(/[[[:space:]]/, "") }
        $1 ~ /^[0-9]+$/ && $2 ~ /^[0-9]+$/ { print $1 "=" $2 }') || refuse "..."
    mods=
    while read -r id; do
        m=$(printf '%s\n' "$map" | awk -F= -v i="$id" '$1 == i && !s++ { print $2 }')
        [[ -n $m ]] || refuse "hl-smi lists no module_id for index $id"
        mods="${mods:+$mods,}$m"
    done <<< "$ids"
    export HABANA_VISIBLE_MODULES=$mods
fi
```

Two scripts surround it. `gaudi_check_cards.sh` runs before training and fails closed on three checks: an **idle baseline** (every allocated card at most 1536 MiB in host `hl-smi` before anything of ours touches it, so a card busy later is ours); **identity** (a `HABANA_VISIBLE_*` set on the host must equal the allocation through the map); a **container probe** (the restriction reaches the container with as many entries as allocated cards). `gaudi_verify_busy.sh` runs in the background and, after 600 s (smoke) or 900 s (long runs), reports whether every allocated card exceeds 1536 MiB. The argument: `idle` before, `busy` after, one process per card (an assumption the scripts state) and as many ranks as cards, so every rank sits on an allocated card.

The threshold is calibrated, and once was wrong. The first one, 4096 MiB, came from the 1-card batch-200 reading (42,940 MiB) plus a wrong guess that Habana reserves a whole card. At 50 images per card a healthy rank read 3,219 MiB, and the verifier, then a kill switch, cancelled a good 4-card job at 15 minutes (64614665, gaudi005). Both scripts now share `GAUDI_IDLE_MAX_MIB=1536` in `gaudi_hlsmi.sh` (twice the idle reading), and the verifier only reports.

The proof came from three runs: smoke 64613789 (gaudi003) had index 3 / module 7 at 42,940 MiB and the other seven cards at 768; job 64614665 had indices 0-3 at 3,219 and 4-7 at 768; job 64617617 had 0-3 at 11,431 to 14,352 and 4-7 at 768. `memory.used` reflects real use, not a whole-card reservation.

Check, for your job. In `slurm-<id>.out`: `idle baseline OK`, then `gaudi.sh: SLURM_JOB_GPUS=3 -> HABANA_VISIBLE_MODULES=7` (your numbers), then `restriction OK`, and 15 minutes in, `card use OK` under an `hl-smi` table with your indices above 1536 MiB. If your model is small, confirm in the smoke that a healthy rank exceeds 1536 MiB. **Failure looks like** `ERROR: ... refusing to launch` from the check (cheap: Oct 4's first refusal took 9 seconds) or a `WARNING: card use check` line, which means inspect and `scancel` by hand.

Step 5: Smoke-test on one card

Do. Run the whole path briefly on one card before any long job. Ark's `gaudi_smoke.sh`, in order:

1. Host checks (weights, image, run directory absent), because they cost seconds here and a slot later; the engine refuses an existing run directory anyway.
2. `hl-smi` plus the `index,module_id,bus_id,memory.used` table, so the log keeps the map and pre-training memory.
3. `gaudi_check_cards.sh`: nothing trains before isolation is confirmed.
4. A strict weight-load check, because a silent partial load passes a smoke while proving nothing.
5. `gaudi_verify_busy.sh 600 &`, the identity evidence.

6. Training under `timeout --kill-after=60 1800`, as a throwaway seed-1 run.
7. A `grep` of the run log for world size, per-rank batch, load messages and `BT=` lines.
8. A verdict: exit 124 is success only if a `BT=` line was logged, because a hung run also ends in 124; 137 means SIGKILL (kill-after or the OOM killer).

UniMiSS+'s `smoke_3d.sh` does the same except step 5 and without a run log: one epoch plus validation, `test.py` on the saved checkpoint, then a summary that exits 1 unless it finds a device line naming `hpu`, matched layers above 0, a finite loss, a per-step time and a finite final metric (it first turns `tqdm`'s carriage returns into newlines). It and `smoke_2d.sh` run only the pre-launch `gaudi_check_cards.sh`: they show that the restriction reached the container and that training ran on `device: hpu`, but not that the allocated card went busy; `gaudi_verify_busy.sh 900` starts only in the long run scripts, `run_3d.sh` and `run_2d.sh`. If your smoke runs longer than about 10 minutes, start the verifier in it too, as Ark's does in step 5.

A passing run (Oct 5): 3D showed `device: hpu` in both scripts, 198 matched layers, loss 0.653475, 2.80 s/it at batch 8 over 25 steps and a finite test line, in 133 s plus 39 s; 2D ran 1.45 s/it at batch 32 over 10 steps in 66 s. Do not read speed from a smoke: first steps and short last batches compile graphs, and 3D's 2.80 s/it became 1.01 s/step in the real run.

A smoke should pass only when the log shows, in this order, a startup line naming `device: hpu` and then `BT=` lines: a CPU fallback (Step 3) still trains, only slowly, and can still log `BT=`. The device match ignores case because Ark logs `Device: hpu`; where your script also prints the torch build, as UniMiSS+ does, match Habana's `2.10.0a0+git...` on that line too. Your version can be this short (placeholder names; Ark names a run directory `<experiment>_seed_<seed>`):

```
#!/usr/bin/env bash
#SBATCH -A grp_jliang12
#SBATCH -p gaudi
#SBATCH -q public
#SBATCH -N 1
#SBATCH --gres=gpu:hl225:1
#SBATCH -c 18
#SBATCH --mem=96G
#SBATCH -t 00:45:00
set -uo pipefail
cd /scratch/$USER/my-fork || exit 1
[[ -f containers/gaudi/gaudi.sif && ! -e my_exp_seed_1 ]] || exit 1
hl-smi -Q index,module_id,memory.used -f csv
./gaudi_check_cards.sh || exit 1
./gaudi_verify_busy.sh 600 &
timeout --kill-after=60 1800 ./experiment_scripts/my_exp/start_my_exp.sh 1 gaudi
rc=$?
log=my_exp_seed_1/train.log
grep -qi 'device: hpu' "$log" || { echo "no hpu device line in $log"; exit 1; }
n=$(grep -c 'BT=' "$log" 2>/dev/null)
[[ ${n:-0} -gt 0 && ( $rc -eq 0 || $rc -eq 124 ) ]] && echo "SMOKE OK" || exit 1
```

Delete the throwaway run directory afterwards, or the next start refuses it.

Step 6: Scale to N cards and tune the data loader

The sbatch header is most of the diff between your A100 run script and its Gaudi counterpart:

Setting	A100 (<code>run_d1_8a_class.sh</code>)	Gaudi (<code>run_d1_8a_gaudi.sh</code>)
Account	<code>class_cse49478170fall2026</code>	<code>grp_jliang12</code>
Partition / QOS	<code>public,htc / class</code>	<code>gaudi / public</code>
Time	<code>07:30:00</code>	<code>7-00:00:00</code>

Setting	A100 (<code>run_d1_8a_class.sh</code>)	Gaudi (<code>run_d1_8a_gaudi.sh</code>)
Cards	<code>--gres=gpu:1</code> <code>--constraint=a100_80</code>	<code>--gres=gpu:hl225:4</code>
CPUs / memory	<code>-c 8 / 64G</code>	<code>-c 72 / 256G</code>
ARK_WORKERS	8	16
Added		<code>PT_HPU_LAZY_MODE=1</code> , card check, busy check, <code>gaudi</code> argument, a guarded <code>cd</code>

ARK_BATCH_SIZE is global; the engine divides it by world size, so 200 means 50 per card on 4 cards. An equal global batch keeps the A100 comparison fair, but per-card batch statistics change and there is no SyncBatchNorm.

The data loader becomes the bottleneck. At 8 workers per rank on 4 cards Ark was data-bound: MIMIC averaged 0.92 s/step with DT (data time) up to 0.8 s, and `ps` on `gaudi009` showed 32 `pt_data_worker` processes at 66-70% CPU and 4 ranks at about 187% each, roughly 29 of 72 cores busy; a cycle took about 49 minutes. At 16 workers per rank (64 on 72 cores) MIMIC reached 0.73 s/step (1.27x) and the cycle about 40.4 minutes, still data-bound: steps not waiting on data took about 0.24 s. ChestX-ray14 got about 12% slower (0.51 to 0.57 s/step) without being data-bound (CPU contention is a guess). Pre-resizing images was not tried: it would change the data path relative to the A100 runs.

A rule of thumb from those numbers (my arithmetic, not a measured optimum): request 18 cores per card, then size workers so that ranks times workers, plus about 2 cores per rank (the measured 187%), fits in `-c`; $4 \times 16 + 4 \times 2 = 72$. If DT stays near BT after that, the run is input-bound (the workers are not keeping up) and more cards will not help. That is not the same as CPU-bound: workers can also wait on shared storage (`/data` reads, the first pass over a dataset) or on another input-pipeline stage, and the run above at 8 workers per rank had only about 29 of 72 cores busy, which argues against pure CPU saturation. Raising workers is the first cheap experiment, not a diagnosis: before asking for more cores or settling on a worker count, look at per-process CPU (`top` or `htop` on the node via `srun --overlap`, as in Step 7, or the job's CPU utilization in `sstat` while it runs and `sacct` after) and at I/O.

Measure speed from elapsed time. Ark logs BT every 50th step; those samples suggested 7-9x an A100 on 4 cards, while elapsed-time averages gave about 2.3x on MIMIC, because the samples missed the stalls (BT spikes of 0.74-6.5 s). Divide pass duration by steps.

Not problems. Each pass's short last batch recompiles the graph: about 18-20 s on 4 cards (job 64620656), a 7.44 s step on 1 card (smoke 64613789), once per shape per job and not cached across jobs, so every resume pays again. Per-card memory varies: 3,219 MiB in one job, 11,431-14,352 MiB in the next, same configuration and point.

Step 7: Submit the long job with a backup link

The partition limit is 7 days. Submit the run, then a resume that waits on it:

```

sbatch run_d1_8a_gaudi.sh           # Submitted batch job <A>
sbatch -d afterany:<A> resume_d1_8a_gaudi.sh  # backup link

```

`afterany` fires however the first job ends (time limit, crash, cancel), and the resume continues from the last checkpoint; if training already finished, Ark's engine refuses to continue past the last cycle and the link ends at once. Chain a build and its smoke with `afterok:<build>` instead: Gaudi jobs can start immediately, and Jarod's first unchained smoke had to be cancelled because it would have started before the image existed.

If you edit the run script mid-training (Ark moved workers from 8 to 16 this way), `scancel` the backup link first, because it was submitted with the old script and fires the moment the running job ends; then cancel the running job, submit a resume and a new backup.

Housekeeping: Slurm writes `slurm-<jobid>.out` in the submit directory unless `--output` is set; Habana's logs land in the bound log directory; `sacct` gives state and elapsed time afterwards (run_3d: COMPLETED, 01:29:01). To inspect a running job's node (`hl-smi`, `ps`, `top`), Jarod opened a shell in the job with `srunk --overlap`; a Ctrl-C'd one stays listed RUNNING until the job ends, which is harmless. Both repos refuse an existing output directory: `mv old runs aside`.

Debugging table

Symptom	Likely cause	Where to look
"Requested node configuration is not available"	No card requested on the gaudi partition	Add <code>--gres=gpu:hl225:1</code> , even for a build
"Invalid qos specification"	Class account, or <code>-q class_gaudi</code>	<code>-A grp_jliang12 -q public</code>
Container reports 8 HPUs on a 1-card job Job launches 8 ranks	Expected: <code>device_count()</code> ignores the restriction <code>--nproc-per-node</code> from <code>device_count()</code>	HABANA_VISIBLE_MODULES in the container; <code>hl-smi</code> memory at 10-15 min Count <code>SLURM_JOB_GPUS</code> entries, as <code>common.sh</code> does
no HABANA_VISIBLE_* reached the container allocated card index N already shows ... MiB in use before launch <code>synStatus=8</code> [Device not found] WARNING: card use check at 15 min Metrics about 9 points low, no error Slow steps, DT close to BT	No mapping (no <code>SLURM_JOB_GPUS</code> , e.g. outside a job) or not forwarded past <code>--cleanenv</code> Leftover process, yours or another user's	Wrapper stderr: <code>SLURM_JOB_GPUS=...</code> <code>-> HABANA_VISIBLE_MODULES=...</code>
<code>synStatus=8</code> [Device not found] WARNING: card use check at 15 min Metrics about 9 points low, no error Slow steps, DT close to BT	An interrupted Gaudi process did not exit Threshold above your per-card memory, or a rank on the wrong card Eager mode (<code>PT_HPU_LAZY_MODE</code> unset or 0 in the container) Data-loader bound	Demo README: find the PID with <code>hl-smi</code> , kill it The <code>hl-smi</code> table above it; <code>scancel</code> by hand if confirmed <code>PT_HPU_*</code> lines in the card check's container probe <code>ps</code> for <code>pt_data_worker</code> CPU; workers per rank, <code>-c</code>
Multi-second step at each pass's end, or slow first steps	Lazy-mode graph compile (new shape)	Expected; read speed after warm-up
Device line says <code>cpu</code> or <code>cuda</code> Freeze shows another torch or numpy, or the build fails on a conflict	Habana import failed; the fallback chose another device A requirement wants different base packages	Run the import inside the image by hand Constraints, <code>--no-deps</code> , or a compatible pin; <code>/opt/pip-freeze.txt</code>
"User not listed in /etc/subuid, trying root-mapped namespace"	<code>--fakeroot</code> fallback	Harmless for a pip-only def; <code>apt-get</code> untested
Run refuses: directory exists	Earlier or smoke run directory	<code>mv</code> it aside or delete the throwaway

Sources

- Lab demo (github.com/jlianglab/Gaudi_Demo, local copy `Gaudi_Demo-main/`): `README.md`, `gaudi-apptainer.def`, `gaudi-apptainer.sh`, `build.sh`, `requirements.txt`, `requirements_nodeeps.txt`,

- scripts/*.sh; in BenchmarkTransformers/: trainer.py, engine.py, main_classification.py; in MedMNIST/: engine.py.
- Jarod’s copy of 2026Fall-CSE507-Ark-Container: in containers/nvidia/: apptainer.def, requirements.txt; in containers/audi/: apptainer.def, requirements.txt, requirements_nodeps.txt; at the root: nvidia.sh, audi.sh, audi_check_cards.sh, audi_hlsmi.sh, audi_verify_busy.sh, audi_smoke.sh, build_audi_image.sh, run_d1_8a_class.sh, run_d1_8a_audi.sh, resume_d1_8a_audi.sh, README.md (“Audi on Sol”); in experiment_scripts/: common.sh; in base/: accelerator.py, trainer.py, joint_training.py, engine.py.
 - Jarod’s UniMiSS+ port (practice3/UniMiSS-code): in sol_audi/: README.md, apptainer.def, requirements.txt, build_image.sh, unimiss_audi.sh, audi_check_cards.sh, smoke_3d.sh, run_3d.sh, smoke_2d.sh, run_2d.sh; the HPU patch to the released scripts against the authors’ commit 68be0a4 (commits 6898e58, 7e877db), including hpu_device.py in UniMiSSPlus/Downstream/.
 - Jarod’s session notes, Oct 4-6 2026: the Audi port of the Ark+ container (Oct 4), the UniMiSS+ HPU patch, image and Sol runs (Oct 5), and the Sol Audi route reference.